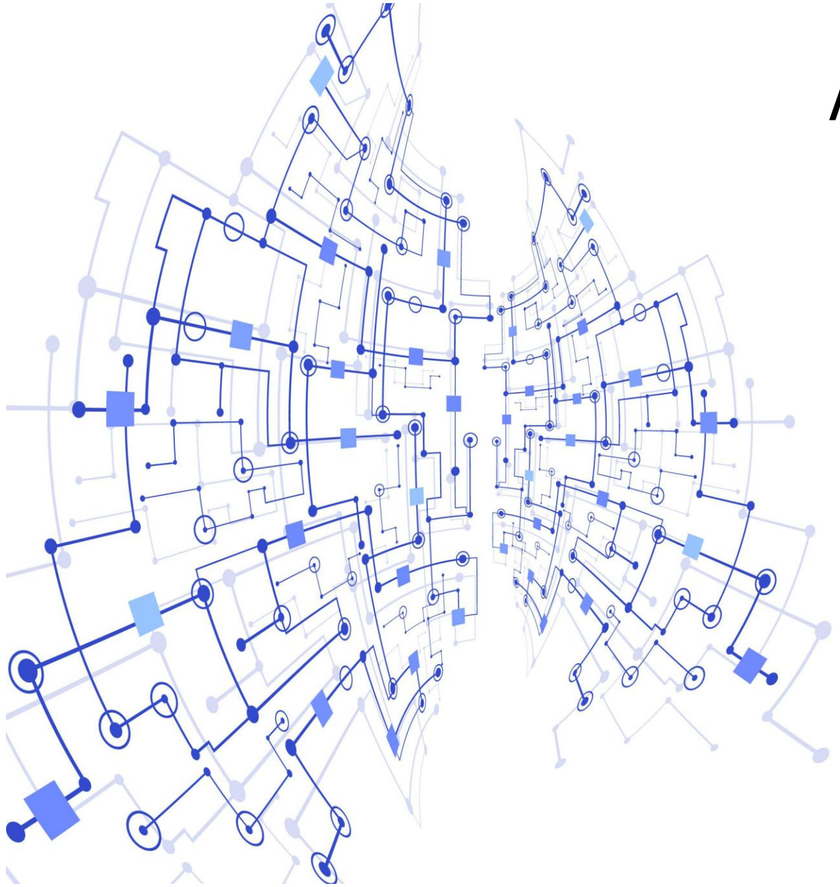


# Algoritma dan Struktur Data

## 5th Meeting

# Queue

Rudy Chandra, S.Kom., M.Kom

**D-III Teknologi Informasi**

**Institut Teknologi Del**

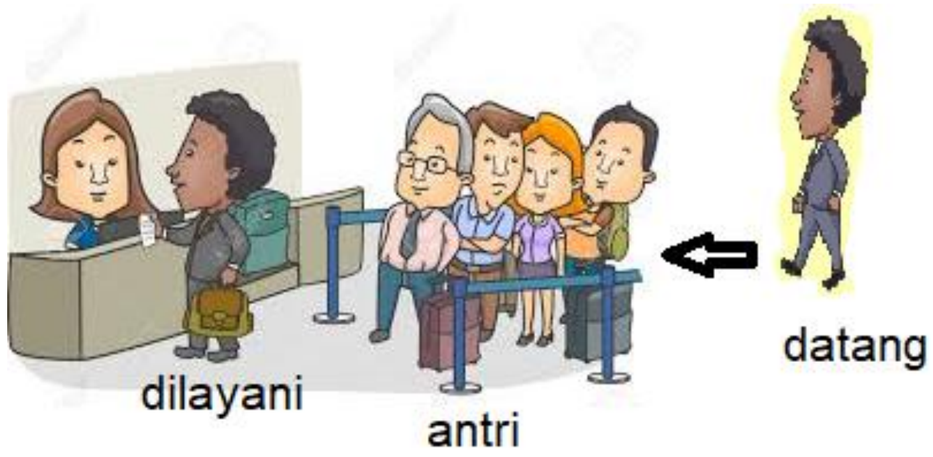
**2025**

# Outline

- Pengertian Queue
- Operasi Dasar Queue
- Pemakaian Queue
- Karakteristik Queue
- Implementasi Queue dengan tabel / array

# Definisi Queue

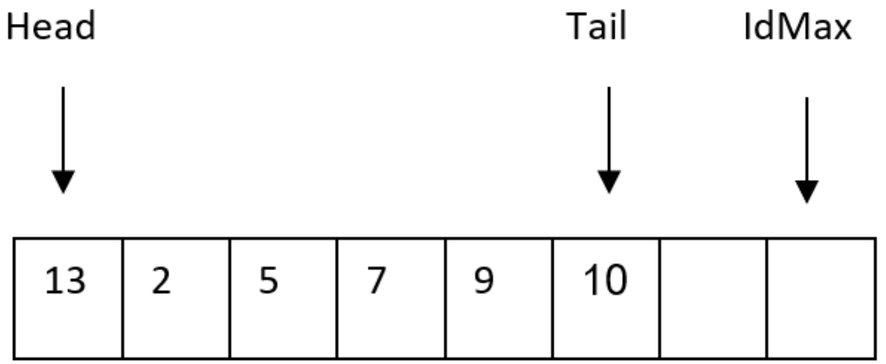
- Queue adalah : kumpulan elemen dengan karakteristik elemen yang **pertama masuk akan keluar pertama kali**, dan sebaliknya elemen yang **terakhir masuk akan keluar terakhir kali**.
- Prinsip pada stack: FIFO (FIRST IN FIRST OUT)
- Queue dikenali elemen "**Head**" dan "**Tail**"
  - Aturan penambahan : selalu pada Tail
  - Aturan pengurangan : selalu pada Head
- Queue Kosong
  - Head = Nil and Tail = Nil



# Operasi Dasar STACK

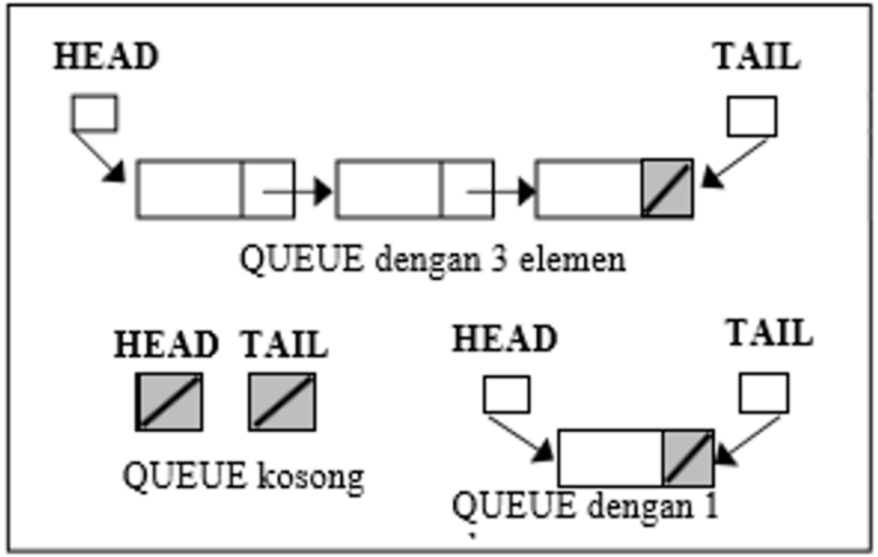
- Add
  - Tambah item ke queue melalui TAIL
- Delete
  - Ambil item dari queue melalui HEAD

# Operasi Dasar Queue




Selanjutnya, anda dapat menambahkan operasi -operasi lain yang anda anggap penting, seperti contohnya:

- **`CreateEmptyQueue`** → untu k membuat Queue
- **`DestroyQueue`** → untuk menghapus Queue
- **`IsQueueEmpty`** → untuk mengetahui apakah Queue kosong
- **`IsQueueFull`** → untuk mengetahui apakah Queue penuh.



# Karakteristik Queue

- Data hanya dapat ***di-insert*** melalui **Tail**
- Data hanya dapat ***di-delete*** melalui **Head**
- Data tidak dapat ***di-insert*** atau ***di-delete*** dari tengah-tengah Queue harus melalui **Tail** dan **Head**

# Pemakaian Queue

- Representasi antrian job yang harus diproses pada system operasi

  [kemungkinan variasi : priority queue]
- Representasi antrian transaksi
- Representasi buffer

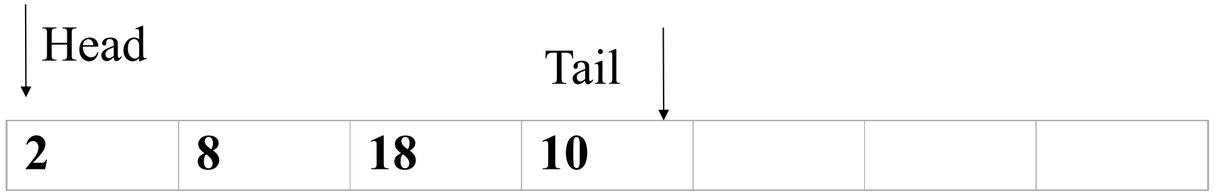
# Variasi Implementasi algoritma

- Versi naif : Head selalu Q.T[1]
  - Penambahan elemen : Tail ++
  - Pengurangan elemen : ambil Head, geser semua elemen dg address Head+1 s/d Tail
- Versi-1 : geser jika perlu
  - Penambahan elemen :

    if tidak penuh && (Tail < IdxMax) then Tail++

    else {geser untuk mendapat tempat}
  - Pengurangan elemen : ambil Head; Head++
- Versi-2 : tabel sirkuler, kue donat, tanpa pergeseran
  - Penambahan elemen: Tail++ atau Tail=1
  - Pengurangan elemen: ambil, Head++ atau Head=1

# Implementasi Alternatif 1 – Versi Naif

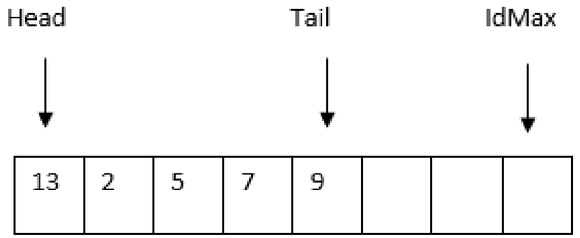



- Apabila nilai 2 akan diambil/dihapus maka HEAD akan bergerak ke indeks nomor 2. Sisa indeks array yang kosong menjadi 4, namun data/nilai yang boleh dimasukan pada antrian hanya 3.
- Untuk mengatasi hal tersebut, dilakukan **proses pergeseran pada antrian**.
- Setiap kali data/nilai dalam antrian diambil/dihapus maka nilai/data dibelakangnya akan maju/bergeser ke indeks nomor 1. Pergeseran ini dilakukan pula oleh data yang dibelakangnya.

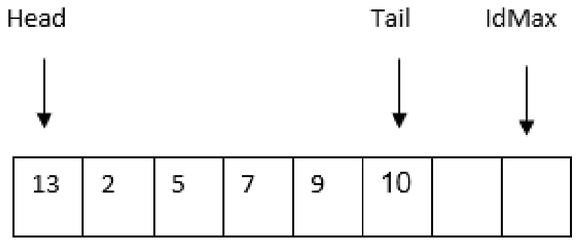
# Implementasi Alternatif 1 – Versi Naif

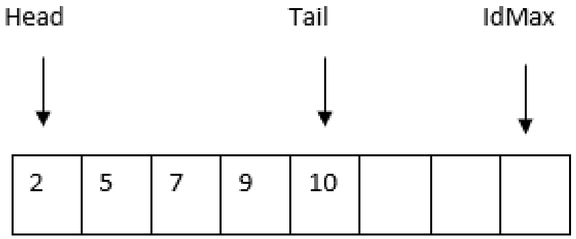
- Head selalu Q.T[1]
- Penambahan elemen : Tail ++
- Pengurangan elemen : ambil Head, geser semua elemen dg address Head+1 s/d Tail

**State-1**



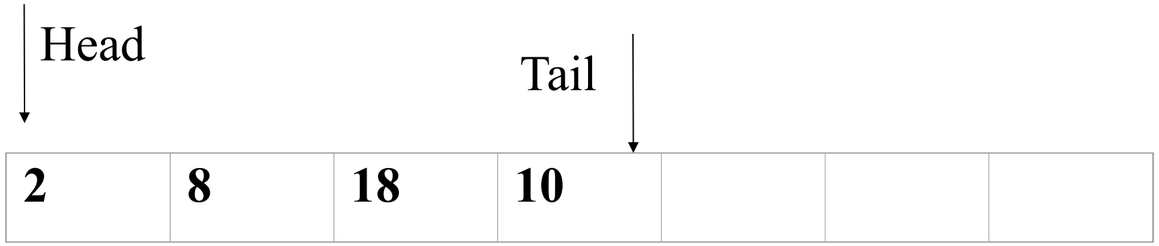

**State-2: Add element**




**State-3:Del element**

# Implementasi Alternatif 1 – Versi Naif

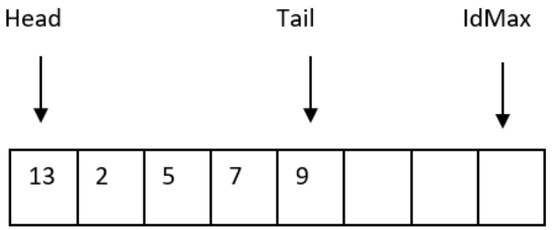



- Pada proses pergeseran tersebut, bagaimana bila antrian memiliki data yang banyak ? Berapa kali proses pergeseran dilakukan ?
- Adakah cara lain untuk mengatasi hal tersebut ?

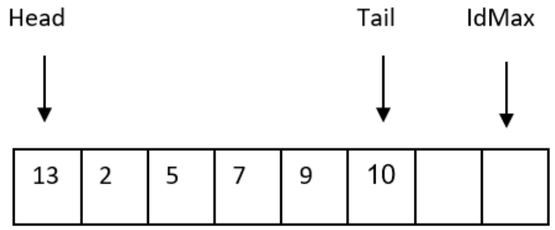
# Impelementasi Alternatif 2 – geser jika perlu

- Penambahan elemen :
  - if tidak penuh && (Tail < IdxMax) then Tail++
  - else {geser untuk mendapat tempat}
- Pengurangan elemen : ambil Head; Head++

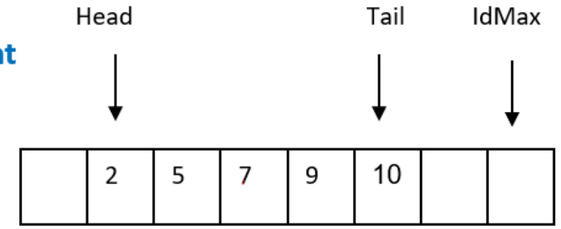
**State-1**

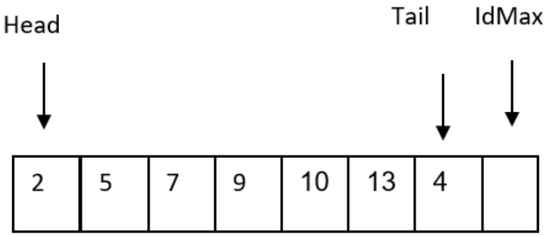



**State-2: Add element**



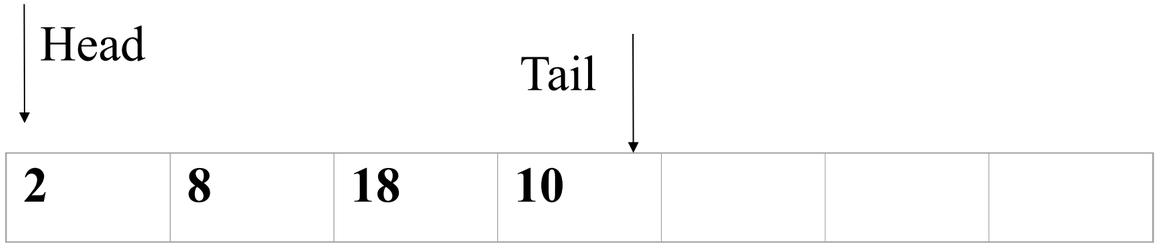

**State-3:Del element**




**State-4:Add Full (geser)**

# Impelementasi Alternatif 2 – geser jika perlu



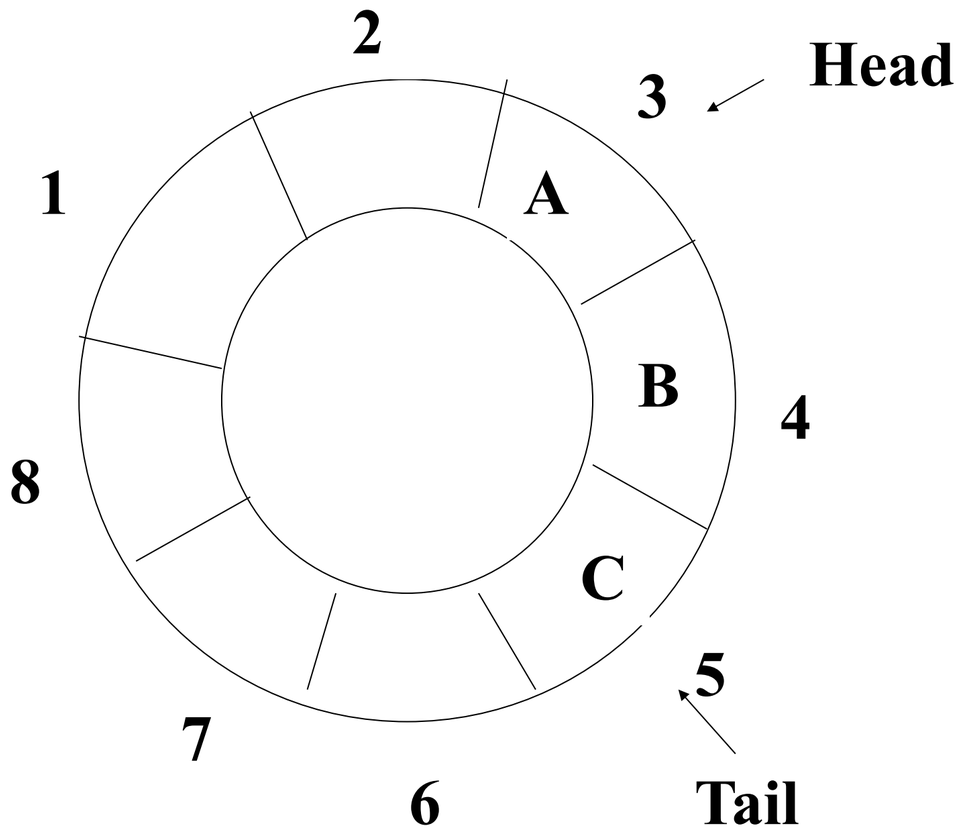

- Bila akan menambah data antrian, data dapat dimasukkan pada posisi (TAIL + 1)
- Bila akan menghapus data antrian, data dapat dihapus pada posisi (HEAD), sehingga posisi HEAD setelahnya adalah (HEAD +1)
- HEAD = TAIL (tetapi bukan nol), berarti data antrian tinggal satu.
- HEAD > TAIL berarti antrian kosong, karena datanya sudah dihapus dan tidak ada lagi data yang dimasukkan
- TAIL = AKHIR INDEKS, berarti antrian penuh

# Impelementasi Alternatif 2 – geser jika perlu




- Apabila suatu saat ditemukan TAIL = AKHIR INDEKS, HEAD berada di posisi indeks nomor : TAIL – 1, apakah hal tersebut termasuk kondisi antrian telah penuh ?
- Apakah kelemahan dari implementasi antrian di atas ?
- Bagaimana solusinya ?

# Impelementasi Alternatif 3 – Sirkular

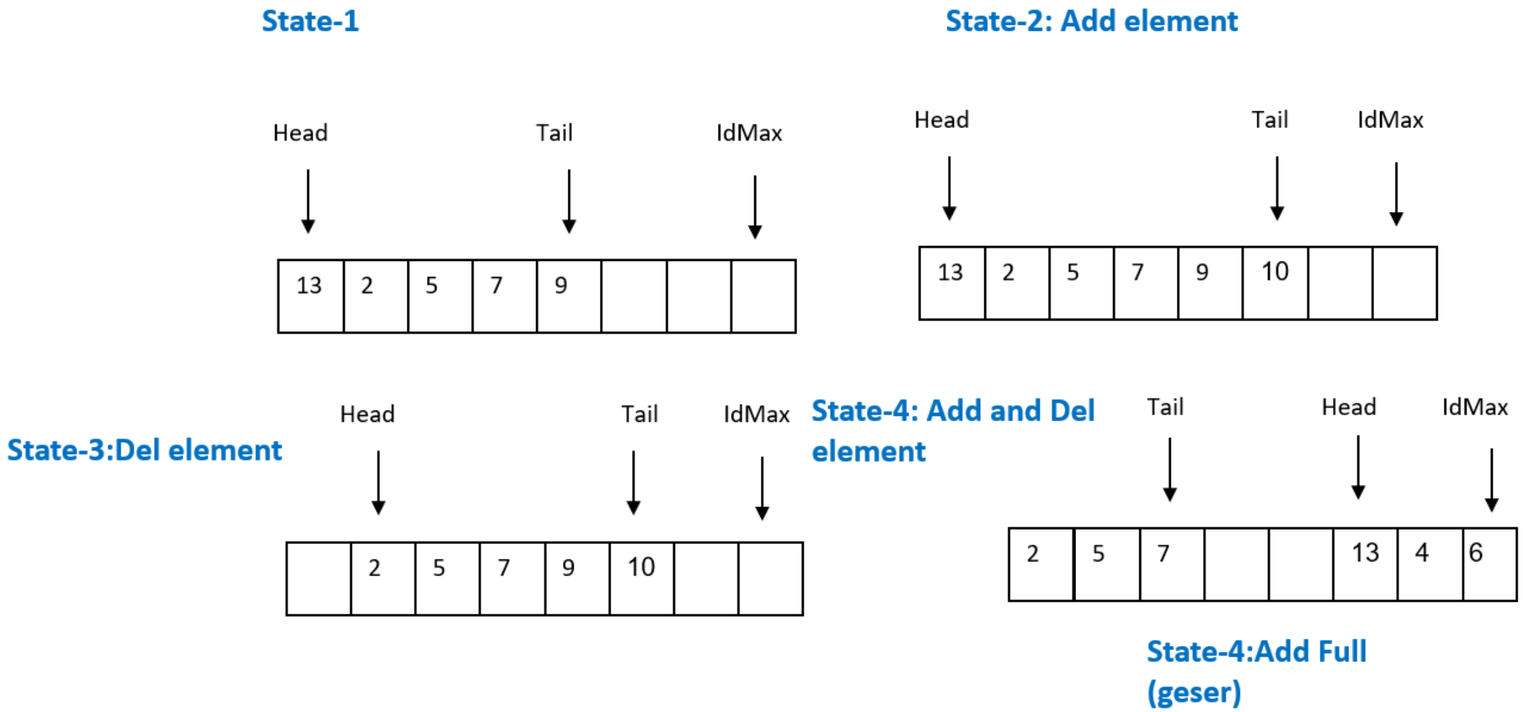



Gambaran array yang seolah-olah ujung kiri dan kanannya disambungkan. Sehingga berbentuk lingkaran/sirkuler.

- Kue donat
- Penambahan elemen: **Tail++ atau Tail=1**
- Pengurangan elemen: **ambil, Head++ atau Head=1**

# Impelementasi Alternatif 3 – Sirkular

- Bila akan menambah data antrian, data dapat dimasukkan pada posisi (TAIL + 1)
- Bila akan menghapus data antrian, data dapat dihapus pada posisi (HEAD), sehingga posisi HEAD setelahnya adalah (HEAD +1)
- HEAD > TAIL berarti antrian kosong, karena datanya sudah dihapus dan tidak ada lagi data yang dimasukkan
- TAIL = AKHIR INDEKS, berarti antrian penuh



# Contoh

queue.h

```h
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

typedef struct {
    int data[MAX];
    int front;
    int rear;
} Queue;

// Inisialisasi Queue
void initQueue(Queue *q);

// Mengecek apakah Queue kosong
int isEmpty(Queue *q);

// Mengecek apakah Queue penuh
int isFull(Queue *q);

// Menambahkan elemen ke dalam Queue (enqueue)
void enqueue(Queue *q, int value);

// Menghapus elemen dari Queue (dequeue)
int dequeue(Queue *q);

// Melihat elemen paling depan
int peek(Queue *q);

// Menampilkan isi Queue
void display(Queue *q);

#endif
```

# Contoh

driver.c

```c
#include <stdio.h>
#include "queue.h"

int main() {
    Queue q;
    initQueue(&q);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    display(&q);

    printf("Peek: %d\n", peek(&q));

    printf("Dequeue: %d\n", dequeue(&q));
    display(&q);

    enqueue(&q, 40);
    enqueue(&q, 50);
    display(&q);

    return 0;
}
```

# Contoh

main.c

```c
#include <stdio.h>
#include "queue.h"

void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(Queue *q) {
    return (q->front == -1);
}

int isFull(Queue *q) {
    return (q->rear == MAX - 1);
}

void enqueue(Queue *q, int value) {
    if (isFull(q)) {
        printf("Queue penuh!\n");
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
    }
    q->rear++;
    q->data[q->rear] = value;
    printf("Enqueue: %d\n", value);
}
```

```c
int dequeue(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue kosong!\n");
        return -1;
    }
    int value = q->data[q->front];
    if (q->front == q->rear) {
        // Queue jadi kosong setelah dequeue
        q->front = q->rear = -1;
    } else {
        q->front++;
    }
    return value;
}

int peek(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue kosong!\n");
        return -1;
    }
    return q->data[q->front];
}

void display(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue kosong!\n");
        return;
    }
    printf("Isi Queue: ");
    for (int i = q->front; i <= q->rear; i++) {
        printf("%d ", q->data[i]);
    }
    printf("\n");
}
```

Run gcc body dan driver sekaligus

Di contoh body = queue.c ; driver = main.c

gcc main.c queue.c -o queue

```
Enqueue: 10
Enqueue: 20
Enqueue: 30
Isi Queue: 10 20 30
Peek: 10
Dequeue: 10
Isi Queue: 20 30
Enqueue: 40
Enqueue: 50
Isi Queue: 20 30 40 50
```

# THANK YOU!